

Course Outline: Preparing Data for Model Training

Title: Preparing Data for Model Training **Skill:** __ — Skill x: Data preparation for training **Learnpack:** + Preparar datos para el entrenamiento de un modelo **File:** s__-w17d49-preparar-datos-para-el-entrenamiento-de- **Prerequisite:** Familiarity with collecting and ingesting data (APIs, scraping, databases, CSVs — covered in previous modules) **Target Audience:** Beginner developers learning to prepare raw data for machine learning models **Total Lessons:** 9 **Format:** Mixed (11% hands-on)

Course Description

Raw data is rarely ready to train a model. Before any learning can happen, data must be extracted, reshaped, cleaned, and delivered to the right destination. This course covers the ETL pipeline — Extract, Transform, Load — with emphasis on the transformation stage: cleaning dirty records, validating schema, normalizing numeric ranges, enriching missing fields, and applying business rules. Students finish by writing a real transformation pipeline in Python and understanding where the clean data should live once it's ready.

Course Philosophy

This course emphasizes:

- Practical data manipulation over theoretical statistics
 - Understanding *why* each transformation step exists — not just how to run it
 - Building habits of data quality that prevent silent bugs in model training
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - ETL and Data Transformation	4
02 - Loading Data	2
03 - Assessment	1
04 - Outro	1
Total	9

00.0 Training Data Starts with Preparation

Learning Objectives:

- Explain why raw data cannot be fed directly to a machine learning model

- Identify the three stages of an ETL pipeline and the role each plays in model preparation

Content Outline:

- The gap between collected data and model-ready data
- Why models trained on dirty data produce unreliable results
- ETL as the bridge: Extract → Transform → Load
- Overview of what this course covers and where it fits in the ML workflow

Transitions: This lesson sets the stage. Everything that follows — transformation techniques, loading targets — solves the problem introduced here.

Key Principles:

- Garbage in, garbage out: model quality is bounded by data quality
- Transformation is the most time-consuming and highest-impact stage of ETL

Examples:

- A dataset of housing prices where some entries have negative square footage — a model trained on this will learn the wrong pattern
 - A churn prediction dataset where 80% of customers have null "last_login" — the model can't learn from what it can't see
-

01.0 ETL and Data Transformation

Learning Objectives:

- Define ETL and describe each stage at a high level
- Explain why the Transform stage is the centerpiece of model preparation
- Describe what sub-topics this section will cover (cleaning, validation, normalization, enrichment) and how they connect

Content Outline:

- What ETL means: Extract (you already know this), Transform (this section's focus), Load (Section 02)
- The Transform stage's job: take raw, inconsistent data and make it useful
- The six transformation operations: cleaning, validation, normalization, enrichment, deduplication, business rules
- How these operations chain together in a real pipeline

Transitions: This overview frames the whole section. Lesson 01.1 goes deep into cleaning and validation — the first two operations in every transformation pipeline.

Key Principles:

- ETL is a reference concept, not a strict technology: any process that extracts, transforms, and loads data qualifies

- The Extract stage was covered in prior modules (APIs, scraping, databases, CSVs) — this course starts where that ended
- Transform is where 80% of engineering effort in data pipelines lives

Examples:

- A raw CSV of product sales → extract from file → transform (clean prices, validate dates, normalize units) → load into a feature store for training
 - A quick mental model: ETL is like a kitchen — ingredients come in (Extract), get prepped (Transform), and go to the table (Load)
-

01.1 Cleaning and Validation

Learning Objectives:

- Apply three core cleaning techniques: handling missing values, removing duplicates, and fixing type errors
- Distinguish between cleaning (fixing what's broken) and validation (rejecting what can't be fixed)
- Write a validation rule that catches schema violations before they enter the training dataset

Content Outline:

- **Cleaning:** What broken data looks like
 - Missing values: drop row, fill with median/mode/constant, or flag with a sentinel
 - Type errors: a price column containing "N/A" strings, timestamps stored as integers
 - Inconsistent formatting: "USA", "U.S.A.", "United States" in the same column
- **Deduplication:** Why duplicates distort model learning
 - Exact duplicates vs near-duplicates
 - How to detect and remove them in Pandas
- **Validation:** Setting a quality gate
 - Schema validation: column names, expected types, allowed ranges
 - Row-level rules: required fields, cross-column constraints
 - What to do with invalid rows: drop, quarantine, or raise an error

Transitions: Cleaning removes what's broken. Validation sets the bar for what's allowed. The next lesson — normalization and enrichment — takes clean, valid data and reshapes it to be more useful for a model.

Key Principles:

- Never silently drop data without logging: track how many rows were removed and why
- Validation rules are business rules expressed in code — they should be readable by a non-engineer

Anti-patterns:

- **Filling all nulls with zero:** zero means something in most numeric columns; it's rarely a correct substitute for missing data

- **Validating after training:** catching schema violations only when the model fails is too late — validate at ingestion

Examples:

```
import pandas as pd

df = pd.read_csv("sales.csv")

# Drop exact duplicates
df = df.drop_duplicates()

# Fill numeric nulls with column median
df["price"] = df["price"].fillna(df["price"].median())

# Type coercion
df["date"] = pd.to_datetime(df["date"], errors="coerce")

# Validation: drop rows where price is negative
invalid = df[df["price"] < 0]
print(f"Dropped {len(invalid)} invalid rows")
df = df[df["price"] >= 0]
```

01.2 Normalization and Enrichment

Learning Objectives:

- Apply min-max scaling and z-score normalization to numeric features
- Encode categorical variables using one-hot encoding
- Enrich a dataset by adding a derived feature that improves model signal
- Apply a business rule to enforce domain-specific constraints

Content Outline:

- **Normalization:** Why raw numeric ranges confuse models
 - Min-max scaling: rescale to [0, 1]
 - Z-score standardization: rescale to mean=0, std=1
 - When to use each
- **Categorical encoding:** Models need numbers, not strings
 - One-hot encoding: turning "red/green/blue" into three binary columns
 - Label encoding: when categories have a natural order (low/medium/high → 0/1/2)
- **Enrichment:** Adding features the raw data doesn't have
 - Derived features: `days_since_last_purchase = today - last_purchase_date`
 - Aggregated features: `avg_purchase_value_per_user`
- **Business rules:** Constraints that must be enforced regardless of the raw data
 - Example: `discount_rate` must be between 0 and 1; clamp anything outside this range
 - Example: minimum order value is \$5; treat anything below as noise

Transitions: After cleaning, validating, and normalizing, the data is ready to be written. The  challenge applies all of these steps in sequence. After that, Section 02 covers where the cleaned data goes.

Key Principles:

- Normalization must be fit on training data only — never on the full dataset — to avoid data leakage
- Enrichment is where domain knowledge pays off: a good derived feature can be more predictive than ten raw ones

Anti-patterns:

- **One-hot encoding high-cardinality columns:** a "country" column with 180 values → 180 new columns → sparse, noisy data
- **Normalizing after train/test split:** fitting the scaler on test data leaks statistics into training

Examples:

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

scaler = MinMaxScaler()
df[["price", "quantity"]] = scaler.fit_transform(df[["price", "quantity"]])

# One-hot encode category
df = pd.get_dummies(df, columns=["category"])

# Derived feature
df["days_since_signup"] = (pd.Timestamp.today() - df["signup_date"]).dt.days

# Business rule: clamp discount
df["discount"] = df["discount"].clip(0, 1)
```

01.3 Transforming a Dataset

Challenge Prompt: Load the provided raw CSV, apply a full transformation pipeline — drop duplicates, fill missing numeric values with the column median, normalize the **price** and **quantity** columns to [0, 1], and save the cleaned result to **clean_data.csv**.

Solution Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv("raw_data.csv")
```

```
df = df.drop_duplicates()
for col in ["price", "quantity"]:
    df[col] = df[col].fillna(df[col].median())

scaler = MinMaxScaler()
df[["price", "quantity"]] = scaler.fit_transform(df[["price", "quantity"]])

df.to_csv("clean_data.csv", index=False)
print(f"Done. {len(df)} rows saved to clean_data.csv")
```

Challenge Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv("raw_data.csv")

# Step 1: remove duplicate rows
df = # your code here

# Step 2: fill missing values in "price" and "quantity" with each column's
median
for col in ["price", "quantity"]:
    df[col] = # your code here

# Step 3: normalize "price" and "quantity" to [0, 1]
scaler = MinMaxScaler()
df[["price", "quantity"]] = # your code here

# Step 4: save the result
df.to_csv("clean_data.csv", index=False)
print(f"Done. {len(df)} rows saved to clean_data.csv")
```

02.0 Loading Data

Learning Objectives:

- Explain what "loading" means in the ETL context and why it is a distinct stage from transformation
- Describe the role loading plays in making transformed data available for model training
- Preview the four types of data targets covered in the next lesson

Content Outline:

- What loading means: persisting transformed data into a system that serves a specific purpose
- Why loading is a separate stage: transformed data in memory is temporary — it must land somewhere durable

- The loading decision: where data goes determines who or what can use it, at what speed, and at what cost
- Four categories of target systems (overview only — details in 02.1): data lake, data warehouse, analytics DB, feature store
- Common loading patterns: batch write, streaming append, upsert

Transitions: This lesson establishes *why* loading matters and *what* the options are. Lesson 02.1 goes deep into each of the four target types — what they store, who uses them, and how to choose.

Key Principles:

- Loading is not an afterthought: choosing the wrong target can make data inaccessible for training or unusable at inference time
- The target system must match the consumer: a data lake serves batch training; a feature store serves real-time inference

Examples:

- A batch ETL pipeline that runs nightly → loads to a data warehouse for analysts and a feature store for the model
- A streaming pipeline → appends events to a data lake in Parquet files for later batch training

02.1 Data Targets

Learning Objectives:

- Compare data lakes, data warehouses, analytics databases, and feature stores across four dimensions: what they store, who uses them, query style, and ML relevance
- Choose the correct target type for a given training scenario
- Explain why a feature store is preferred over a raw data warehouse for model serving

Content Outline:

- **Data Lake**
 - What it stores: raw and semi-processed data in any format (CSV, Parquet, JSON, images)
 - Who uses it: data engineers, batch ML training jobs
 - Query style: batch (Spark, Athena, Hive)
 - ML relevance: training on large historical datasets; cheap storage; slow retrieval
- **Data Warehouse**
 - What it stores: structured, aggregated, business-ready data
 - Who uses it: analysts, BI tools, reporting
 - Query style: SQL
 - ML relevance: good for features derived from aggregations; not designed for raw training data
- **Analytics Database**
 - What it stores: structured data optimized for fast analytical queries (column-oriented)
 - Who uses it: data analysts, product teams
 - Examples: ClickHouse, DuckDB, BigQuery

- ML relevance: useful for feature engineering queries; faster than a warehouse for ad-hoc analysis
- **Feature Store**
 - What it stores: pre-computed, versioned features ready for model training and serving
 - Who uses it: ML engineers, model training pipelines, inference services
 - ML relevance: the most ML-native target — same features used for training are served at inference, preventing training-serving skew
 - Examples: Feast, Hopsworks, Tecton

Transitions: With the transformation done and the target chosen, the ETL pipeline is complete. The assessment checks understanding of the full pipeline.

Key Principles:

- Training-serving skew is a silent killer: using different feature logic for training and inference produces models that fail in production — feature stores prevent this
- A data lake is the cheapest option but requires the most work to query; a feature store is the most expensive but the most model-ready

Anti-patterns:

- **Training directly from a production database:** query contention, no versioning, no reproducibility
- **Building your own feature store prematurely:** use managed solutions (Feast, Hopsworks) until scale demands otherwise

Examples:

- Startup with limited budget: raw data → S3 data lake → Pandas for training
- Mid-size ML team: raw data → data lake → feature engineering → Feast feature store → model training + serving

03.0 Data Preparation Assessment 🧠

Learning Objectives:

- Demonstrate understanding of all ETL transformation operations
- Identify correct loading targets for given ML scenarios
- Recognize common anti-patterns in data preparation

Question Format: Multiple choice

Questions:

1. A numeric column has 15% missing values. Which approach is most appropriate for a training dataset?
 - a) Drop the entire column
 - b) Fill with the column median and continue

- c) Train the model anyway — it will handle nulls internally
 - d) Replace with zero
2. You're normalizing features for a classification model. You fit the scaler on the full dataset before splitting into train and test. What problem does this cause?
- a) Nothing — fitting on the full dataset gives better normalization
 - b) Data leakage: test data statistics influence the training transformation
 - c) The scaler will throw an error on the test set
 - d) The model will overfit to the training data
3. Which transformation step removes records that violate schema or business constraints?
- a) Normalization
 - b) Enrichment
 - c) Validation
 - d) Deduplication
4. A "country" column has 200 unique values. One-hot encoding it would create 200 new binary columns. What is the likely outcome?
- a) Better model accuracy due to more information
 - b) Sparse, high-dimensional input that degrades model performance
 - c) No effect — tree-based models handle this automatically
 - d) The encoder will refuse to process more than 100 categories
5. You need the same feature values used at training time to also be available at inference time, without recomputing them. Which target system is designed for this?
- a) Data lake
 - b) Data warehouse
 - c) Analytics database
 - d) Feature store
6. Which of the following is an example of data enrichment?
- a) Replacing missing prices with the column median
 - b) Clamping discount values to [0, 1]
 - c) Adding a `days_since_signup` column derived from `signup_date`
 - d) Converting string dates to datetime objects
7. A model trained on a production database performs well in testing but fails after deployment. What is the most likely cause?
- a) The model was not normalized correctly
 - b) Training-serving skew: features differ between training and inference
 - c) The production database is too slow to query
 - d) The model's hyperparameters were not tuned

Evaluation Criteria:

- 7/7: Excellent — full command of ETL pipeline concepts
- 5–6/7: Good — minor gaps in loading targets or normalization pitfalls
- 3–4/7: Needs review — revisit transformation operations and target selection
- Below 3/7: Restart recommended — core ETL concepts not yet solid

Score Interpretation: Each question maps to a key concept: missing values (Q1), leakage (Q2), validation (Q3), encoding (Q4), feature stores (Q5), enrichment (Q6), training-serving skew (Q7). Review the section corresponding to any missed question.

04.0 Your Pipeline Is Ready

Content Outline:

- Course recap: what students accomplished
 - Understood the ETL pipeline and the three stages
 - Learned six transformation operations: cleaning, validation, normalization, enrichment, deduplication, business rules
 - Built a transformation pipeline in Python
 - Chose the right loading target for a given scenario
- Connection to bigger picture: clean, well-structured data is the foundation that determines model ceiling — no amount of model tuning overcomes bad data
- Next steps: the next module uses this prepared data to train a classification model
- Resources for further exploration:
 - pandas documentation: data manipulation and cleaning
 - scikit-learn preprocessing: scalers, encoders
 - Feast documentation: open-source feature store
 - "Designing Machine Learning Systems" (Chip Huyen) — chapters on feature engineering
- Encouragement: data preparation is unglamorous but it's where most ML projects succeed or fail — mastering it gives you an edge most developers skip

Note: This is conclusion only — no theory, exercises, or assessment.
